



MANUAL TECNICO

El presente proyecto consiste en la implementación de un intérprete para un subconjunto del lenguaje GoLite utilizando las herramientas JFlex y Java CUP. El sistema realiza análisis léxico, sintáctico y semántico, además de ejecutar instrucciones mediante un Árbol de Sintaxis Abstracta (AST).

El proyecto fue desarrollado en Java utilizando el entorno NetBeans

Tecnologías Utilizadas

- Java

```
C:\Users\Usuario>java --version
java 26.0.1 2026-04-21
Java(TM) SE Runtime Environment (build 26.0.1+8-34)
Java HotSpot(TM) 64-Bit Server VM (build 26.0.1+8-34, mixed mode, sharing)
C:\Users\Usuario>
```

- JFlex
- Java CUP
- Swing
- Git y GitHub

Arquitectura General

El sistema está dividido en los siguientes módulos:

- Analizador Léxico
- Analizador Sintáctico
- AST
- Entorno de Ejecución
- Manejo de Errores
- Interfaz Gráfica

El flujo general de ejecución es:

Archivo Fuente → Lexer → Parser → AST → Ejecución → Consola

Analizador Léxico

El análisis léxico fue implementado utilizando JFlex.

Archivo principal:

Lexer.flex

Responsabilidades:

- Reconocer palabras reservadas.
- Reconocer identificadores.
- Reconocer números enteros y decimales.
- Reconocer cadenas y caracteres.
- Ignorar espacios y comentarios.
- Reportar errores léxicos.

Analizador Sintáctico

El análisis sintáctico fue implementado utilizando Java CUP.

Archivo principal:

Parser.cup

Responsabilidades:

- Validar la estructura gramatical.
- Construir el AST.
- Recuperarse ante errores sintácticos.
- Generar objetos de ejecución.

Producciones principales:

- Declaraciones
- Asignaciones
- Expresiones
- If
- For
- Println

Entorno de Ejecución

El entorno almacena todas las variables declaradas durante la ejecución.

Clase:

Entorno.java

Funciones principales:

guardar() Almacena una variable dentro del entorno.

buscar() Obtiene una variable previamente declarada.

actualizar() Modifica el valor de una variable existente.

Tabla de Símbolos

Clase:

`Simbolo.java`

Responsabilidad:

Representar una variable del programa.

Atributos principales:

- Identificador
- Tipo
- Valor

Asignacion.java

Permite modificar el valor de una variable existente.

Ejemplo:

`a = 20`

Funciones:

- Buscar la variable.
- Actualizar su valor.

Compuesta.java

Implementa:

`+= -= ++ --`

Ejemplo:

`a += 5`

Funciones:

- Obtener valor actual.
- Realizar operación.
- Actualizar variable.

Println.java

Implementa la instrucción:

`fmt.Println()`

Funciones:

- Evaluar expresiones.
- Evaluar más de una expresión dentro del `println`
- Mostrar resultados en consola.

If.java

Implementa estructuras condicionales.

Funciones:

- Evaluar condición.
- Ejecutar bloque verdadero.
- Ejecutar bloque `else`.

For.java

Implementa ciclos `for`.

Funciones:

- Ejecutar inicialización.
- Evaluar condición.
- Ejecutar actualización.
- Ejecutar bloque de instrucciones.

Expresiones

Operacion.java

Clase encargada de evaluar:

- Operaciones aritméticas
- Operaciones relacionales
- Operaciones lógicas

Ejemplos:

$a + b$

$a > b$

$a \&\& b$

AccVariable.java

Permite acceder al valor de una variable almacenada en el entorno.

Ejemplo:

`fmt.Println(a)`

Manejo de Errores

ReporteE.java

Clase encargada de almacenar errores detectados.

Tipos de errores:

- Léxicos
- Sintácticos
- Semánticos

La información se muestra posteriormente en la interfaz gráfica.